

G10 · Memory Caching — 고정 메모리 벽을 체크포인트로 뚫기

이 권은 arXiv:2602.24281 「Memory Caching: RNNs with Growing Memory」(Behrouz, Li, Deng, Zhong, Razaviyayn, Mirrokni —Titans/Miras/NL 과 같은 Google 라인) 를 다룬다. G02 에서 배운 “fast weight = 고정 크기 압축 메모리” 의 가장 아픈 약점 — 문맥이 길어지면메모리가 넘쳐 recall 이 무너진다 — 을 정면으로 공격하는 논문이다. G07(Nested Learning) 의 CMS, G08(Sleep) 의 consolidation 과 나란히 놓고 보면 “메모리를 여러 개 두는” 세 번째 방식이 보인다.

핵심 RNN 메모리가 넘치는 이유는 하나의 고정 상태에 전부 덮어쓰기 때문이다. Memory Caching(MC) 은 시퀀스를 세그먼트로 자르고, 세그먼트가 끝날 때마다 메모리 상태의 스냅샷(체크포인트) 을 캐시한다. 읽을 때는 현재 메모리 하나가 아니라 캐시된 과거 메모리들에도 같은 query 를 던져 합친다. 유효 메모리가 시퀀스 길이에 따라자라고, 비용은 RNN 의 $O(L)$ 과 attention 의 $O(L^2)$ 사이 ($O(NL)$) 를보간한다.

10.1 무엇이 문제였나 — 고정 메모리의벽

G02·G03 에서 봤듯 이 계열의 메모리는 행렬 하나 ($d \times d$) 또는 작은 MLP 의 weights 다. 크기가 고정이니 서빙 관점에서는 축복이지만 (문맥과무관한 상수 state), 정보 이론 관점에서는 저주다 — d^2 개의 파라미터에시퀀스 전체를 밀어 넣으면 언젠가는 서로 다른 기억이 같은 자리를두고 싸운다 (간섭). 그래서 recurrent 모델은 recall 집약과제 (needle-in-a-haystack, MQAR) 에서 Transformer 에 진다.

비유 칠판이 하나뿐인 강의실이다. 수업이 길어지면 앞부분 판서를 지우고 그 위에써야 한다. attention 은 “칠판을 절대 지우지 않는 대신 칠판을 무한히 사오는”(KV 캐시, 비용 이차) 방식이고, MC 는 “칠판이 다 차기 전에사진을 찍어 두고 새로 쓰는” 방식이다. 사진 (캐시) 은 늘어나지만칠판 (온라인 메모리) 은 하나다.

10.2 어떻게 — 세그먼트, 체크포인트, 그리고 “읽기의변화”

시퀀스를 세그먼트 $S^{(1)}, \dots, S^{(N)}$ 로 나눈다. 메모리 갱신 규칙 f 는 그대로다 — linear attention 이면 외적 누적, Titans 면 surprise 기반 gradient 스텝. 바뀌는 것은 두 가지뿐이다.

1. 세그먼트가 끝나면 그 시점의 메모리 상태를 캐시한다: $M_{L^{(1)}}^{(1)}, M_{L^{(2)}}^{(2)}, \dots$
2. 읽기가 바뀐다: 표준 RNN 의 $y_t = M_t(q_t)$ 대신,

$$y_t = \text{Agg} \left(\underbrace{\{M_{L^{(i)}}^{(i)}(\cdot)\}_{i < s}}_{\text{캐시들}}; \underbrace{M_t^{(s)}(\cdot)}_{\text{온라인 메모리}}; q_t \right)$$

기호 풀이 $M_t^{(s)}$ = 현재 (온라인) 메모리 — 지금도 갱신되고 있는 그 상태. $M_{L^{(i)}}^{(i)} = i$ 번째 세그먼트가 끝난 순간 찍어 둔 스냅샷 (읽기 전용). Agg = 캐시들의 forward 결과를 합치는 방법 (다음 절의 4 가지). N = 세그먼트 수 — **이게 다이얼이다**: $N=1$ 이면 그냥 RNN, $N=L$ (token 마다 캐시) 이면 논문이 보이듯 softmax attention 과 닮은 형태가 복원된다.

직관 갱신 (쓰기) 은 건드리지 않고 **읽기만 바꿨다**는 게 우아한 지점이다. 쓰기는 여전히 $O(L)$ recurrent — 병렬화 · 서빙 성질이 그대로다. 읽기에서만 “과거 체크포인트들에 같은 질문을 던지는” 비용 $O(N)$ 이 추가된다.

10.3 캐시를 합치는 4 가지 방법

① Residual Memory — 그냥 다더한다

$y_t = M_t^{(s)}(q_t) + \sum_i M_{L^{(i)}}^{(i)}(q_t)$. 메모리가선형 (행렬) 이면 수학적으로는 고정 메모리로 붕괴한다 (행렬들을 미리 합쳐두면 되니까). 그런데도 실험에선 성능이 오른다 — 잔차 합이 먼과 거로의 **retention** 통로 역할을 하기 때문.

② Gated Residual Memory (GRM) — 관련 있는 캐시만 세게듣는다

입력 의존 게이트 $\gamma_t^{(i)} = \text{softmax}(\langle u_t, \text{MeanPool}(S^{(i)}) \rangle)$ 로 각 캐시의 기여를 조절한다. $u_t = x_t W_u$ 는 “지금 token 이 어떤 과거를 찾는가” 를 나타내는 connector. **게이트가 입력 의존이라 미리 합쳐둘수 없고**, 선형 메모리에서도 붕괴하지 않는다 — 캐시가 진짜 캐시로 살아난다.

③ Memory Soup — 출력이 아니라 파라미터를 섞는다

같은 게이트로 캐시들의 **weight 자체를 가중 평균**해 그 token 전용 메모리 M_t^* 를 즉석에서 만들고 $y_t = M_t^*(q_t)$. 선형이면 GRM 과 동치 (선형성 때문에 “weight 섞고 query” = “각각 query 하고 출력 섞기”). **deep/비선형 메모리에서 갈라진다** — token 마다 자기만의 비선형 회수 함수를 조립하는 셈.

④ Sparse Selective Caching (SSC) — MoE 처럼 top-k 만 부른다

router 가 관련도 $r_t^{(i)} = \langle u_t, \text{MeanPool}(S^{(i)}) \rangle$ 로 상위 **k 개 캐시만** 골라 읽는다. MeanPool 은 사전 계산되므로 선택은 싸고 병렬적이며, **선택된 캐시만 accelerator 로 로드**하면 된다. 논문의 효율 실험에서 “성능은 대등 이상, 오버헤드는 최소” 로 4 가지 중 최선의 균형.

시스템 모델링 관점 SSC 를 보는 순간 기시감이 들어야 한다 — 이걸 **fast-weight** 체계의 **paged/selective attention** 이다. KV 캐시 서빙에서 페이지 단위 로드 + top-k retrieval 을 하듯, 여기서는 “파라미터 덩어리 (스냅샷)”가 페이지다. 상태크기 회계: 캐시 하나 = 메모리 모듈 하나 크기 (예: Titans 급 $2rD^2b$). N 개 세그먼트면 per-user 상태가 N 배 — G09 에서 했던 “state 가자란다” 문제가 **fast-weight** 에도 돌아온 것이다. 대신 SSC 덕에 HBM 상주는 top-k 개만이면 되고, 나머지는 값싼 계층 (→ MEMOIR 의 HBF·warm tier) 에 둘 수있다. “read-only 스냅샷 + top-k 로드”는 우리가 G09 에서 논한 frozen-expert-on-HBF 와 정확히 같은 배치 패턴이다.

10.4 체크포인트인가, 독립압축기인가

캐시를 만들 때 숨은 설계 선택이 있다. 세그먼트 s 의 메모리를 **이전 세그먼트의 마지막 상태에서 이어** 시작할것인가 ($M_0^{(s)} = M_{L^{(s-1)}}^{(s-1)}$ — “하나의 학습 과정의 체크포인트”), 아니면 **독립 초기값에서 새로** 시작할것인가 (“세그먼트별 독립 압축기”).

- **체크포인트 관점**: token = 훈련 샘플, 캐시 = 최적화 궤적의 스냅샷. 과거 지식이 자연스럽게 이월되지만, 스냅샷들이 서로 상관되어있다.
- **압축기 관점**: 각 캐시가 그 세그먼트만의 깨끗한 요약. 간섭이 없지만 세그먼트 간 연속성은 잃는다.

논문의 ablation 결론: **각자 장단이 있다** — 정답이 하나가 아니라 과제 의존적이다.

주의 이 구분은 G08(Sleep) 의 consolidation 을 생각할 때 중요해진다. “무엇을 teacher 로 삼아 느린 메모리로 옮길 것인가”에서, 체크포인트 계열 (상관된 스냅샷들)은 평균·souping 이 자연스럽고, 독립 압축기 계열은 개별 이관이 자연스럽다. G11(NSTM)에서 이 선택이 실제로 성패를 가르는 것을 보게 된다.

10.5 복잡도와 효율 — $O(L)$ 과 $O(L^2)$ 사이의 다이얼

N (세그먼트 수)	정체	token 당 읽기 비용	유효 메모리
1	표준 RNN	$O(1)$	고정
중간	MC	$O(N)$ (SSC 면 $O(k)$)	N 배로 성장
L	attention 유사	$O(L)$	전체 보존

세그먼트 분할도 다이얼이다 — 균일 분할 대신 **로그 분할** (최근은 잘게, 먼 과거는 굵게) 을 쓰면 캐시 수가 $O(\log L)$ 로 늘리면서 최근 해상도는 유지된다. 실험 (760M/1.3B, FineWeb 30B/100B tokens) 에서 MC 는 언어 모델링·NIAH·MQAR·LongBench 전반에서 base recurrent 를 끌어올리고, in-context recall 에서 **Transformer** 와의 격차를 좁히며 최신 recurrent 들을 앞선다 (최고는 여전히 Transformer).

10.6 NM 계열 안에서의 자리 — “메모리를 여러 개 두는” 세 번째방법

이 시리즈에서 메모리를 여러 개 두는 방법이 이제 셋이 되었다.

방법	측	무엇이 여러 개인가	권
CMS (Nested Learning)	주파수	갱신 주기가 다른 메모리 레벨들	G07
Sleep 의 expert 확장	수명	sleep 마다 늘어나는 low-rank expert	G08
Memory Caching	시간 (이력)	같은 메모리의 과거 스냅샷들	G10

CMS가 “빠른/느린 메모리를 나란히” 두고, Sleep이 “느린 쪽에 새 용량을 증설” 한다면, MC는 “한 메모리의 역사를 보존” 한다. 세 측은 직교한다 — 실제로 NSTM(G11)은 TTT 메모리에 MC를 얹었고, CMS 레벨마다 MC를 얹는 조합도 자연스럽다.

핵심 MC의 존재 의의를 한 줄로: “fast weight의 유한 용량 문제를, 갱신규칙을 바꾸지 않고 ‘읽기 대상의 개수’로 푼다.” 용량이 부족하면 더 세게 압축하는 게 아니라, 압축본을 여러 장 갖고 골라 읽는 것이다.

10.7 한계와 정직한주의

- **recall 왕좌는 여전히 Transformer** — MC는 격차를 좁힐 뿐 역전하지 못한다 (논문 스스로 명시).
- 캐시는 **per-user** 상태를 N배로 키운다 — 서버 메모리 회계가 다시 어려워진다 (SSC·계층 배치로 완화).
- 게이트·router가 MeanPooling 같은 **단순한 문맥 대표**에 의존한다 — 저자들도 더 표현력 있는 pooling/routing을 future work로 남겼다.
- 스냅샷 평균 (Soup)이 **언제 안전한가**는 이 논문이 답하지 않는다 — 그 답의 결정적 단서가 G11(NSTM)의 ablation에 있다.